

RELATÓRIO AULA 007 MUNIF

Apresentação dos objetivos gerais da aula, ressaltando como a avaliação correta de expressões e o controle rigoroso do fluxo de execução previnem bugs críticos em produção, ilustrando com casos reais da indústria (como o "goto fail" da Apple, o problema do ano bissexto no Zune e a vulnerabilidade de coerção de tipos no PHP).

Quiz inicial proposto aos alunos para testar conhecimentos prévios e palpites sobre casos de borda em precedência, coerção implícita de tipos, avaliação de expressões relacionais encadeadas e regras de avaliação booleana em linguagens como Python, Java, JavaScript e C.

Objetivo e ligação com o mundo real

Objetivo

O objetivo desta aula é entender **como as linguagens avaliam expressões e controlam o fluxo de execução**, para que o seu código faça exatamente o que parece fazer, **sem surpresas** de precedência, conversão ou desvio.

Para o engenheiro de software, isso significa:

- ler código alheio sem ser enganado pela indentação ou pela precedência
- evitar conversões implícitas perigosas (== do JS e do PHP)
- escolher a estrutura de controle mais legível para cada caso

Quando o fluxo sai do controle

2014 · "goto fail" (Apple)

Uma linha `goto fail`; duplicada pulava a verificação de assinatura: iOS e macOS aceitavam certificados falsos.

2008 · Tocadores Zune (Microsoft)

No último dia de um ano bissexto, um `while` nunca terminava, e todos os aparelhos travaram.

PHP · comparação com ==

`"0e4..." == "0e8..."` é verdadeiro: logins que comparavam hashes com == aceitavam senhas erradas.

Entendimento sobre como a aula tende a seguir e os conceitos relacionados a ela, visto que, tende a resolver erros que são comuns do dia a dia, por falta de costume ou entendimento sobre a linguagem em questão

Quiz de abertura: o que cada linha imprime?

Anote o seu palpite para cada trecho **antes** de executar. Vamos conferir ao longo da aula.

1 · Python

```
print(-2 ** 2, 2 ** 3 ** 2)
```

2 · Java

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

3 · JavaScript

```
console.log("5" - 2, "5" + 2);
```

4 · C

```
printf("%d\n", 3 > 2 > 1);
```

5 · Python

```
x = 0
print(x or "padrão", 3 > 2 > 1)
```

6 · JavaScript

```
console.log(0 == "", "" == "0", 0 == "0");
```

palpites imagino que o que será impresso em ordem numérica será: “-4, 512”, java 3”3”9, javascript realiza a soma 3,7, a linguagem C não imprimirá nada, python imprime x, javascript imprime false, true, true

```
main.py x +
1 print(-2 ** 2, 2 ** 3 ** 2)
2
```

Conso

-4 512

Em Python, o operador de exponenciação (**) tem precedência maior que o menos unário, portanto $-2 ** 2$ é avaliado como $-(2 ** 2) = -4$. Além disso, a exponenciação possui associatividade à direita, logo $2 ** 3 ** 2$ é avaliado como $2 ** (3 ** 2) = 2 ** 9 = 512$.

```
J Main.java x +
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         System.out.println(1 + 2 + "3" + 4 + 5);
6     }
7 }
8 }
```

Co

3345

Em Java, a avaliação ocorre da esquerda para a direita. Primeiramente ocorre a soma aritmética $1 + 2 = 3$. Em seguida, ao somar com a string "3", ocorre a concatenação, resultando em "33". Os números subsequentes 4 e 5 também são concatenados como texto, gerando "3345".

```
index.js x +
console.log("5" - 2, "5" + 2);
```

Cc

3 52

Em JavaScript, a subtração obriga a conversão da string "5" para o número 5, resultando em $5 - 2 = 3$. Já o operador de adição realiza a concatenação quando um dos operandos é string, resultando em $"5" + 2 = "52"$.

```
C Main.c x +
1 #include <stdio.h>
2 int main()
3 {
4     printf("%d\n", 3 > 2 > 1);
5 }
```

0

Em C, a comparação relacional encadeada $3 > 2 > 1$ é avaliada da esquerda para a direita. Primeiro $3 > 2$ resulta em verdadeiro (1). Em seguida, compara-se $1 > 1$, o que é falso (0). Portanto, a saída impressa é 0.

OneCompiler Upgrade AI Python Run

```
main.py x +
1 print(x or "padrão", 3 > 2 > 1)
2
```

Console I/O AI Agent

Traceback (most recent call last):
File "/box/main.py", line 1, in <module>
print(x or "padrão", 3 > 2 > 1)
^
NameError: name 'x' is not defined

Ao tentar executar a expressão em Python sem definir previamente a variável 'x', o interpretador lança um erro de NameError ('x' is not defined), interrompendo a execução antes da avaliação lógica do operador 'or'.

```
JS index.js × +
1 console.log(0 == "", "" == "0", 0 == "0");
2
true false true
```

No JavaScript, o operador de igualdade ampla (==) realiza coerção implícita de tipos: `0 == ""` é verdadeiro (pois ambos coagem para 0); `"" == "0"` é falso (são duas strings com conteúdos diferentes); e `0 == "0"` é verdadeiro (a string é convertida para número). Isso demonstra que a igualdade ampla não é transitiva.

Expressões aritméticas: questões de projeto

- Quais são as regras de **precedência**?
- Quais são as regras de **associatividade**?
- Em que **ordem os operandos** são avaliados?
- Há restrições a **efeitos colaterais**?
- O usuário pode **sobrecarregar** operadores?
- Que **mistura de tipos** é permitida?

Lisp dispensa a precedência (p. 306)

(+ a (* b c)): notação prefixa, e os parênteses dizem a ordem.

a + b * c com a=3, b=4, c=5

da esquerda para a direita

(3 + 4) * 5 = 35

da direita para a esquerda

3 + (4 * 5) = 23

com precedência (* antes de +)

3 + 4 * 5 = 23

APL: sempre da direita para a esquerda (p. 304)

3 x 4 + 5 = 27

Visão geral das questões fundamentais de projeto na avaliação de expressões aritméticas, comparando como linguagens distintas tratam regras de precedência, associatividade e notações alternativas (como Lisp e APL).

Quais são as regras de precedência?

As regras de precedência determinam a ordem em que diferentes operadores são avaliados em uma expressão.

Ordem padrão: Na maioria das linguagens, operadores de maior precedência (como multiplicação e divisão) são executados antes de operadores de menor precedência (como adição e subtração), resultando em $3 + (4 * 5) = 23$.

Alternativas:

Lisp: Dispensa a precedência tradicional utilizando notação prefixa e parênteses explícitos para definir a ordem exata ((+ a (* b c))).

APL: Não utiliza precedência convencional; as expressões são avaliadas sempre da direita para a esquerda ($3 \times 4 + 5 = 27$).

Quais são as regras de associatividade?

A associatividade define a ordem de agrupamento e avaliação para operadores adjacentes que possuem o mesmo nível de precedência.

Da esquerda para a direita: Agrupa as operações iniciais à esquerda (ex: $(3 + 4) * 5 = 35$).

Da direita para a esquerda: Agrupa as operações iniciais à direita (ex: $3 + (4 * 5) = 23$).

Em que ordem os operandos são avaliados?

Refere-se à ordem em que os operandos de um operador (ou os argumentos de uma função) são calculados.

Enquanto a maioria das linguagens avalia os operandos da esquerda para a direita, algumas permitem que o compilador altere essa ordem para fins de otimização.

Há restrições a efeitos colaterais?

Um efeito colateral ocorre quando a avaliação de uma expressão modifica o valor de alguma variável ou estado (por exemplo, o uso de funções com mutação de estado ou operadores de incremento como `x++` dentro de uma expressão).

Algumas linguagens impõem restrições rigorosas sobre onde ou quando efeitos colaterais podem ocorrer para evitar que o resultado da expressão mude dependendo da ordem de avaliação dos operandos.

O usuário pode sobrecarregar operadores?

Indica se a linguagem permite que o programador redefina o significado de operadores existentes (como `+`, `-`, `*`) para operar com tipos de dados definidos pelo usuário (como matrizes, números complexos ou objetos personalizados), recurso presente em linguagens como C++ e Python, mas ausente em outras.

Que mistura de tipos é permitida?

Trata de como a linguagem lida com operações envolvendo tipos de dados diferentes (por exemplo, somar um número inteiro com um número de ponto flutuante).

Pode envolver coerção (conversão implícita automática realizada pelo compilador/interpretador) ou exigir tipagem estrita, onde qualquer mistura de tipos exige uma conversão explícita (*cast*) por parte do programador.

Precedência e associatividade na prática

Expressão	Python	JavaScript	Java
<code>-2 ** 2</code>	-4	erro de sintaxe	(não há <code>**</code>)
<code>2 ** 3 ** 2</code>	512	512	—
<code>1 + 2 + "3"</code>	<code>TypeError</code>	"33"	"33"
<code>-7 / 2</code> (inteiros)	-4 (com <code>//</code>)	-3.5	-3
<code>-7 % 2</code>	1	-1	-1

Associatividade (p. 303)

Esquerda: `*` `/` `%` `+` `-`
Direita: `**` (Python, JS, Ruby, Fortran),
`++`, `--`, menos unário
Visual Basic: `^` à esquerda!

Excel

`=-2^2` dá **4**: lá o menos unário vence o `^`.

Na dúvida, **use parênteses** (p. 305): eles deixam a intenção explícita para o compilador e para o próximo leitor.

é possível verificar como as associações mudam de acordo com as linguagens como por exemplo a validação de se operadores estão lado a lado e tem a mesma precedência cada linguagem escolhe a sua maneira, e podemos vir isso no slide.

Ordem dos operandos e efeitos colaterais

Exemplo do livro (p. 307)

```
int a = 5;

int fun1(void) {
    a = 17;      /* efeito colateral */
    return 3;
}

int main(void) {
    a = a + fun1(); /* 8 ou 20? */
    printf("a = %d\n", a);
}
```

8

Java

a ordem é garantida: da esquerda para a direita (p. 308)

20

C (gcc 14)

a ordem não é definida pelo padrão

20

Go

a especificação não define a leitura de `a` em relação à chamada

Transparência referencial (p. 308): sem efeitos colaterais, a ordem deixa de importar, e $f(x) + f(x)$ pode virar $2 * f(x)$.

Análise do efeito colateral: quando uma função altera o estado de variáveis globais ou externas durante a avaliação (como `fun1` modificando `'a'`), o resultado final da expressão passa a depender da ordem de avaliação dos operandos praticada pela linguagem enquanto o Java garante a avaliação da esquerda para a direita (resultando em 8), linguagens como C e Go não definem uma ordem estrita para a leitura de `'a'` antes/depois da chamada da função (resultando em 20).

Operadores sobrecarregados

- O mesmo símbolo com **vários significados**: + soma inteiros, reais e concatena (Java)
- Risco (p. 309): & em C++ é E bit a bit e "endereço de"
- **Definida pelo usuário** (p. 404): C++, C#, Python, Rust, Kotlin. Java **não** permite

Java

```
System.out.println('a' + 1);           // 98
System.out.println("" + 'a' + 1);     // a1
x.add(y).multiply(tres);               // BigDecimal: sem +
```

Python · sobrecarga com métodos especiais

```
class Vetor:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, outro):          # v1 + v2
        return Vetor(self.x + outro.x,
                       self.y + outro.y)

    def __mul__(self, k):               # v * 3
        return Vetor(self.x * k, self.y * k)

v = Vetor(1, 2) + Vetor(3, 4)         # Vetor(4, 6)
print(v * 3)                          # Vetor(12, 18)
```

Análise mostra que operadores podem realizar diferentes ações sem mudar sua forma, possuindo uma sobrecarga, a mesma presente no Java, dessa forma podendo concatenar strings ou realizar a soma propriamente dita

Conversões de tipos: alargamento, estreitamento e coerção

Java · coerção e cast

```
int grande = 123456789;
float f = grande;           // alargamento implícito
System.out.println((int) f); // 123456792 !

double d = 3.99;
System.out.println((int) d); // 3 (cast)
System.out.println((byte) 200); // -56

byte b = 10, c = 20;
// byte s = b + c; // erro: b + c é int (p. 312)

System.out.println(1 / 2); // 0
System.out.println(1 / 2.0); // 0.5
```

Linguagem	int → real (implícito)	int32 + int64
C	sim	coerção
Java	sim (alargamento)	coerção para long
Python	sim	(um só tipo int)
Go	não: float64(i)	erro de compilação
Rust	não: i as f64	erro de compilação
ML, F# (p. 312)	não	erro de compilação

Coerção esconde erros (p. 312): em `d = b * a`; com a digitado por engano no lugar de c, o Java converte e compila.

Coerção implícita: JavaScript e PHP

JavaScript

```
"5" - 2           // 3
"5" + 2           // "52"
"5" * "2"         // 10
true + 1          // 2
[] + {}           // "[object Object]"
null + 1          // 1
undefined + 1     // NaN
"b" + "a" + "a" + "a" // "baNaNa"

0 == ""          // true
"" == "0"        // false
0 == "0"         // true (não é transitivo!)
0 === "0"        // false (=== não converte)
```

PHP · "type juggling"

```
var_dump("0e123" == "0e456"); // true !
var_dump("0e123" === "0e456"); // false
var_dump("1" == "01");        // true

md5("240610708") // "0e462097431906..."
md5("QNKCDZO")   // "0e830400451993..."
// == entre os dois hashes: true
```

Regra prática

Em JavaScript e PHP, compare com `===` e `!==`, que não fazem coerção.

Expressões relacionais e booleanas

C · booleano é inteiro (p. 315)

```
int a = 3, b = 2, c = 1;
printf("%d\n", a > b > c); // 0 !
// (3 > 2) vale 1, e 1 > 1 é falso

printf("%d\n", (a > b) && (b > c)); // 1

if (-1) printf("verdadeiro\n"); // != 0
```

Python · comparações encadeadas

```
a, b, c = 3, 2, 1
print(a > b > c) # True: (a > b) and (b > c)

# contam como FALSO em um if:
# 0 0.0 "" [] {} None
# contam como VERDADEIRO:
# "0" [0] " "
```

Linguagem	O que o if aceita
C, Python, JavaScript	qualquer valor (0, vazio e nulo contam como falso)
Java, C#, Go, Rust	somente booleano: <code>if (n)</code> com <code>n</code> inteiro nem compila

Exemplificação dos operadores de sobrecarga: o mesmo símbolo (como `+`) assume múltiplos comportamentos dependendo dos tipos dos operandos. Linguagens como C++, Python e Rust permitem que o usuário sobrecarregue operadores para tipos personalizados, enquanto Java restringe a sobrecarga apenas a recursos nativos da linguagem.

Avaliação em curto-circuito

Java · && faz curto-circuito, & não

```
// busca do livro (p. 316)
while (i < lista.length && lista[i] != chave)
    i++;           // && evita lista[lista.length]

String s = null;
s != null && s.length() > 0 // false
s != null & s.length() > 0 // NullPointerException
```

JavaScript · ?. e ??

```
const u = { nome: "Ana", endereco: null };
u.endereco?.cidade // undefined, sem erro

const qtd = 0;
qtd || 10           // 10 (o 0 se perdeu!)
qtd ?? 10           // 0 (só troca null/undefined)
```

Cuidado com efeitos colaterais (p. 316)

Em `(a > b) || (b++ / 3)`, o `b++` só acontece quando `a <= b`. Em Python, `0 or "padrão"` devolve "padrão": `and` e `or` devolvem um dos operandos.

Exemplos de conversões de tipo (alargamento e estreitamento) e coerção implícita. Enquanto linguagens como C e Java realizam coerções automáticas em certas operações, linguagens modernas e estritamente tipadas como Go e Rust exigem conversões explícitas (cast) para evitar erros silenciosos.

Sentenças de atribuição

C · a atribuição é uma expressão (p. 319-320)

```
sum = count = 0;           // múltiplos alvos
a = b + (c = d / b) - 1;    // exemplo do livro

while ((ch = getchar()) != EOF) { ... }

if (x = y) { ... } // compila! atribui e testa y
```

Linguagem	Atribuição é expressão?
C, C++, JavaScript	sim: <code>if (x = y)</code> compila
Java, C#	sim, mas o <code>if</code> exige booleano
Python	não, exceto o operador morsa <code>:=</code>
Go	não: nem <code>x++</code> é expressão
Rust	vale <code>()</code> : <code>if x = 5</code> não compila

Python · atribuição múltipla (p. 320)

```
a, b = b, a # troca sem temporária
if (n := len(dados)) > 5: ... # morsa (Python 3.8+)
```

Linguagens funcionais (p. 321)

`val` (ML) e `let` (F#, Rust) dão nome a um valor, que não muda.

Demonstração dos perigos da coerção implícita em JavaScript e PHP (type juggling). Nesses cenários, comparações complexas com `==` podem gerar comportamentos não intuitivos e brechas de segurança em validações. A regra prática é utilizar sempre os operadores de igualdade estrita (`===` e `!==`).

Atribuição de modo misto

Java · só alargamento (p. 322)

```
double d = 3;           // ok: alargamento
// int i = 3.7;         // erro: estreitamento
int i = (int) 3.7;      // 3 (cast explícito)
long l = i;             // ok

i += 3.7;                // COMPILA: cast escondido
System.out.println(i);  // 6
```

C · coerção em qualquer direção

```
int i = 3.7;           // 3, em silêncio
char c = 300;          // 44 (só um aviso)
unsigned int u = -1;    // 4294967295
```

Armadilha clássica

`double taxa = acertos / total * 100;` com inteiros dá **0.0**: a divisão inteira acontece **antes** do alargamento para `double`.

Análise das expressões relacionais e booleanas: diferença no tratamento de valores booleanos como inteiros (como em C) versus tipos booleanos estritos (Java/Go/Rust), além de ilustrar como Python aceita comparações encadeadas (`a > b > c`) e avalia a falsidade de objetos/coleções vazias.

Estruturas de controle: quantas são necessárias?

Sequência

uma sentença após a outra

```
a = 1;
b = a + 2;
```

Seleção

escolher um caminho

```
if (x > 0) ...
else ...
```

Iteração

repetir enquanto uma condição vale

```
while (x > 0)
    x--;
```

Böhm e Jacopini (1966): sequência, seleção e laço lógico bastam para qualquer algoritmo. O `goto` não é necessário.

Por que então tantas outras (`for`, `switch`, `do`, `foreach`)? **Facilidade de escrita**. O limite é a **legibilidade**: quanto maior a linguagem, menos do código alheio você reconhece (p. 329).

Explicação da avaliação em curto-circuito: em expressões condicionais lógicas (`&&`, `||`), o segundo operando só é avaliado se o primeiro não for suficiente para determinar o resultado final. O uso incorreto de operadores sem curto-circuito (como `&` em Java) pode causar exceções como `NullPointerException`.

Seleção de dois caminhos: o "else pendente"

C · a endentação engana (p. 331)

```
int sum = 0, count = 5, result = -1;
if (sum == 0)
    if (count == 0)
        result = 0;
else
    result = 1;           // de qual if é este else?
printf("%d\n", result);  // 1
```

Regra de Java, C, C++ e C#

O else pertence ao if **mais próximo** que ainda não tem else. A endentação é ignorada (exceto em Python e F#).

Chaves obrigatórias eliminam o problema: Perl, Go e Rust (p. 332).

Seletores que devolvem valor

```
let y = if x > 0 { x } else { 2 * x };           // Rust: if é uma expressão (como em F#, p. 334)
y = x if x > 0 else 2 * x                       # Python: expressão condicional (p. 306)
```

Exemplificação prática de avaliação em curto-circuito em Java e JavaScript: em expressões com operadores lógicos (como && ou ||), a avaliação é interrompida assim que o resultado final é determinado, prevenindo erros como NullPointerException. Adicionalmente, ilustra-se o uso do operador de encadeamento opcional (?.) e do operador de coalescência nula (??) para lidar de forma segura com valores null e undefined.

Estudo das sentenças de atribuição: análise das diferenças entre linguagens onde a atribuição é tratada como uma expressão que devolve valor (permitindo atribuições encadeadas ou atribuição dentro de condições em C/C++) e linguagens funcionais ou de tipagem mais rígida onde a atribuição é apenas uma instrução.

Laços controlados por contador

C · o for é um laço lógico disfarçado

```
for (int i = 0; i < 10; i++) {
    printf("%d ", i);
    i += 2;           // altera a contagem
}                    // 0 3 6 9

for (; i <= 100; )    // expressões opcionais
    soma += i++;
```

Python · o for percorre um iterável

```
for i in range(0, 10):
    print(i, end=" ")
    i += 2           # NÃO muda nada
                    # 0 1 2 3 4 5 6 7 8 9

for i in range(10, 0, -3):  # 10 7 4 1
    ...
```

Questões de projeto (p. 342)

Qual é o tipo e o escopo da variável? Posso alterá-la no corpo? Os parâmetros são avaliados uma vez ou a cada volta?

Sobre os laços controlados por contador é possível identificar como eles se alteram de acordo com a linguagem e o que será feito, por exemplo no primeiro print o item está sendo incrementado após a entrada no çado e por uma variável dentro do laço, já o segundo exemplo demonstra como a iteração ocorre por conta da uma definição anterior de range que, também, realiza iteração de acordo com a função de python

Laços lógicos, saídas e iteradores

Java · break rotulado (p. 348)

```
externo:
for (int i = 0; i < m.length; i++)
  for (int j = 0; j < m[i].length; j++)
    if (m[i][j] < 0) {
      System.out.println(i + "," + j);
      break externo; // sai dos DOIS laços
    }
```

JavaScript · for...in × for...of

```
const notas = [10, 20, 30];
let total = 0;
for (const i in notas) total += i;
console.log(total); // "0012": chaves!

total = 0;
for (const v of notas) total += v;
console.log(total); // 60: valores
```

Zune (2008) · o laço lógico sem progresso

```
while (dias > 365) {
  if (bissexto(ano)) {
    if (dias > 366) { dias -= 366; ano++; }
    // dias == 366: nada muda, laço infinito!
  } else { dias -= 365; ano++; }
}
```

Pós-teste (do-while) executa **ao menos uma vez**, e isso é fácil de esquecer (p. 347).

Análise sobre laços lógicos e iteradores: destaca a importância do progresso explícito em laços condicionais para evitar loops infinitos (como a falha do Zune), além do uso de rotulagem de laços em Java (break rotulado) para sair de estruturas aninhadas e a distinção no JavaScript entre 'for...in' (que itera sobre chaves) e 'for...of' (que itera sobre valores).

Desvio incondicional: o goto

"goto fail" (Apple, 2014) · versão simplificada

```
int valida_certificado(void) {
  int err;
  if ((err = verifica_hash()) != 0)
    goto fail;
  goto fail; // duplicada: desvia SEMPRE
  if ((err = verifica_assinatura()) != 0)
    goto fail; // nunca é alcançada
fail:
  return err; // 0 = "certificado válido"
}
```

Dijkstra (1968)

"Go To Statement Considered Harmful"

Quem tem goto hoje

C, C++, C#, Go. Java não tem (p. 353)

Gotos restritos

break, continue e return melhoram a legibilidade (p. 353)

Com chaves obrigatórias (Go, Rust) ou com gcc -Wall (aviso de endentação enganosa), o bug teria sido pego.

Análise do risco dos desvios incondicionais (comando goto), ilustrada pelo clássico erro de segurança "goto fail" da Apple em 2014, onde uma instrução duplicada ignorava a verificação de assinatura digital. O slide contrapõe a visão histórica de Dijkstra com o design de linguagens modernas que evitam o uso de goto, exigindo blocos estruturados com chaves ou oferecendo apenas saltos rotulados controlados (break, continue, return).

Comandos protegidos de Dijkstra

```
if x >= y -> max := x
[] y >= x -> max := y
fi
```

Exemplo do livro (p. 354): se $x = y$, qualquer ramo serve.

- Todas as **guardas** são avaliadas
- Várias verdadeiras: escolha **não determinística**
- Nenhuma verdadeira: erro (no `if`) ou fim do laço (no `do ... od`)
- Base do CSP de Hoare (1978) e do `select` de Go

Go · `select`, herdeiro dos comandos protegidos

```
a <- "a"      // os dois canais
b <- "b"      // estão prontos

select {      // Go sorteia uma guarda
case x := <-a:
    contagem[x]++
case x := <-b:
    contagem[x]++
}

// 1000 repetições:
// map[a:505 b:495] (muda a cada execução)
```

Análise detalhada do risco dos desvios incondicionais (comando `goto`), ilustrada pelo clássico erro de segurança "goto fail" da Apple em 2014, onde uma instrução duplicada ignorava a verificação de assinatura digital. O slide contrapõe a visão histórica de Dijkstra ("Go To Statement Considered Harmful") com o design de linguagens modernas que evitam ou restringem o uso de `goto`, exigindo blocos estruturados com chaves ou oferecendo apenas saltos rotulados controlados (`break`, `continue`, `return`).

Discussão das regras de atribuição em modo misto e coerção entre tipos numéricos de precisões diferentes. Destaca-se a armadilha clássica da divisão inteira antes da atribuição para ponto flutuante, que resulta na perda de precisão antes mesmo do alargamento do tipo.

Exercício em duplas: encontre o bug (1/3)

Para cada trecho: (a) qual é a saída? (b) qual regra causa o problema? (c) como corrigir?

1 · JavaScript

```
const precos = [10, 20, 30];
let total = 0;
for (const p in precos)
    total += p;
console.log("Total: " + total);
```

2 · Python

```
def desconto(preco, d=None):
    d = d or 10    # padrão: 10%
    return preco * (100 - d) / 100

print(desconto(200, 0))
```

Análise do teorema de Böhm e Jacopini sobre estruturas de controle fundamentais (sequência, seleção e iteração), mostrando que construções adicionais (`for`, `switch`, `do-while`) existem para facilitar a escrita e melhorar a legibilidade do código.

Exercício: encontre o bug (2/3)

Para cada trecho: (a) qual é a saída? (b) qual regra causa o problema? (c) como corrigir?

3 · Java

```
int dia = 2; String nome = "";
switch (dia) {
    case 1: nome = "domingo";
    case 2: nome = "segunda";
    case 3: nome = "terça";
    default: nome = "inválido";
}
System.out.println(nome);
```

4 · C

```
int saldo = 100, saque = 50;
if (saque > saldo);
    printf("saldo insuficiente\n");
printf("fim\n");
```

Análise do problema do "else pendente" em estruturas de seleção encadeadas. Em linguagens como C, C++ e Java, o 'else' associa-se automaticamente ao 'if' mais próximo que não possui um 'else', independentemente da endentação do código. Linguagens modernas como Go, Rust e Perl resolvem essa ambiguidade tornando o uso de chaves obrigatório.

Exercício: encontre o bug (3/3)

Para cada trecho: (a) qual é a saída? (b) qual regra causa o problema? (c) como corrigir?

5 · Go

```
x := 10
if x > 5 {
    x := x * 2
    fmt.Println("dentro:", x)
}
fmt.Println("fora:", x)
```

6 · Java

```
int acertos = 7, total = 10;
double taxa = acertos / total * 100;
System.out.println(taxa + "%");
```

Apresentação dos comandos protegidos de Dijkstra, um modelo não determinístico onde todas as guardas são avaliadas simultaneamente. Se mais de uma for verdadeira, a escolha do caminho é feita aleatoriamente, servindo de base conceitual para construções como o comando 'select' em Go.

Síntese: armadilhas e como as linguagens ajudam

Tema	Armadilha	Como as linguagens ajudam
Precedência	<code>-2 ** 2, 1 + 2 + "3"</code>	parênteses; JS obriga <code>(-2) ** 2</code>
Operandos	<code>a + f()</code> com efeito colateral (C, Go)	Java fixa a ordem; funções puras
Conversões	coerção silenciosa (C, JS, PHP)	Go e Rust exigem conversão explícita; <code>===</code>
Booleanos	<code>0, "", []</code> são falsos; <code>a > b > c</code> em C	<code>if</code> só com booleano (Java, Go, Rust); <code>??</code>
Atribuição	<code>if (x = y)</code>	atribuição não é expressão (Python, Go, Rust)
Seleção	<code>switch</code> sem <code>break</code> ; <code>else</code> pendente	<code>-></code> , <code>match</code> exaustivo, chaves obrigatórias
Laços	<code>for...in</code> no JS; <code>laço sem progresso</code>	<code>for...of</code> , iteradores, <code>break</code> rotulado
Desvio	<code>goto</code> sem restrição	<code>break</code> , <code>continue</code> , <code>return</code> : <code>gotos</code> restritos

Exercício Encontre o Bug (1/3)

1. JavaScript (for...in em arrays)

(a) Saída: "Total: 0012"

(b) Regra: O laço 'for...in' em JavaScript itera sobre as chaves/índices do objeto/array como strings ("0", "1", "2"), e não sobre os valores numéricos. Ao somar com 'total' (inicializado em 0), ocorre coerção de tipo e concatenação de strings.

(c) Correção: Utilizar 'for...of' para iterar sobre os valores do array: 'for (const p of precos) { total += p; }'.

2. Python (argumento padrão com valor falso)

(a) Saída: 180.0

(b) Regra: Em Python, o número 0 é avaliado como Falso em um contexto booleano. Ao passar `d=0`, a expressão '`d = d or 10`' avalia '`0 or 10`', resultando em 10 (aplicando 10% de desconto em vez de 0%).

(c) Correção: Checar explicitamente por None: '`if d is None: d = 10`'.

Exercício Encontre o Bug (2/3)

3. Java (fall-through no switch)

(a) Saída: "inválido"

(b) Regra: O comando 'switch' tradicional em Java possui comportamento de 'fall-through'. Como não há comandos 'break' ao final de cada 'case', a execução continua executando todas as atribuições subsequentes até o bloco 'default'.

(c) Correção: Adicionar 'break;' ao final de cada 'case' ou utilizar a sintaxe moderna do switch (Java 14+): '`case 2 -> nome = "segunda";`'.

4. C (ponto e vírgula após condição de if)

(a) Saída:
saldo insuficiente

fim

(b) Regra: O ponto e vírgula imediatamente após a condição do 'if' ('if (saque > saldo);') encerra o corpo da estrutura condicional como uma instrução nula. O bloco printf subsequente é executado incondicionalmente.

(c) Correção: Remover o ponto e vírgula após a instrução 'if': 'if (saque > saldo) printf(...)'.

Exercício Encontre o Bug (3/3)

5. Go (shadowing de variável)

(a) Saída:

dentro: 20

fora: 10

(b) Regra: O uso do operador de declaração curta 'x := x * 2' dentro do bloco 'if' cria uma nova variável local 'x' que oculta (shadowing) a variável 'x' do escopo externo. A alteração não afeta a variável externa.

(c) Correção: Utilizar o operador de atribuição simples em vez de declaração: 'x = x * 2'.

6. Java (divisão inteira antes de atribuição para double)

(a) Saída: 0.0%

(b) Regra: Como 'acertos' e 'total' são inteiros, a operação 'acertos / total' realiza uma divisão inteira ($7 / 10 = 0$) antes de multiplicar por 100 e converter para double na atribuição.

(c) Correção: Garantir que ao menos um dos operandos seja ponto flutuante: 'double taxa = (double) acertos / total * 100;' ou 'double taxa = acertos * 100.0 / total;'.

Síntese do Conteúdo

Resumo das principais armadilhas tratadas nas linguagens de programação e os mecanismos modernos desenvolvidos para mitigá-las, abrangendo precedência, efeitos colaterais, coerção, avaliação booleana, comandos de seleção e controle de fluxo.